

searches through manuals for industrial construction equipment, it's likely that `crane operator` will be a more relevant, useful suggestion than `crane feather`. The simplest way to determine which $n + 1$ -gram is the best suggestion is to offer the one that appears most often in our corpus.

Thus, our full algorithm: a user searches for an n -gram, we find all $n + 1$ -grams whose first n elements match the user's n -gram, and we recommend the matching $n + 1$ -gram that appears most frequently in the corpus.

Finding Candidate $n + 1$ -grams

In order to find the $n + 1$ -grams that will constitute our search suggestions, we need to know how long the user's search term is. Suppose the search term is `life is a`, meaning that we're looking for suggestions for how to complete the phrase "life is a . . .". We can use the following simple lines to get the length of our search term:

```
from nltk.tokenize import sent_tokenize, word_tokenize
search_term = 'life is a'
split_term = tuple(search_term.split(' '))
search_term_length = len(search_term.split(' '))
```

Now that we know the length of the search term, we know n —it's 3. Remember that we'll be returning the most frequent $n + 1$ -grams (4-grams) to the user. So we need to take into account the different frequencies of different $n + 1$ -grams. We'll use a function called `Counter()`, which will count the number of occurrences of each $n + 1$ -gram in our collection.

```
from collections import Counter
counted_grams = Counter(grams[search_term_length - 1])
```

This line has selected only the $n + 1$ -grams from our `grams` variable. Applying the `Counter()` function creates a list of tuples. Each tuple has an $n + 1$ -gram as its first element and the frequency of that $n + 1$ -gram in our corpus as its second element. For example, we can print the first element of `counted_grams`:

```
print(list(counted_grams.items())[0])
```

The output shows us the first $n + 1$ -gram in our corpus and tells us that it appears only once in the entire corpus: